

Art Exhibition Management System (AEMS)



Prepared by
Abrar Makki, Brian Daza, and Aditi Ghosh
CS 442
at the
University of Illinois Chicago
Spring 2026 - Group 12

Table of Contents

List of Figures.....	3
List of Tables.....	4
I Project Description.....	5
1 Project Overview.....	5
2 Project Domain.....	5
3 Relationship to Other Documents.....	5
4 Naming Conventions and Definitions.....	7
4a Definitions of Key Terms.....	7
4b UML and Other Notation Used in This Document.....	8
4c Data Dictionary for Any Included Models.....	8
II Project Deliverables.....	8
1 First Release.....	9
2 Second Release.....	10
3 Third Release.....	13
4 Comparison with Original Project Design Document.....	13
III Testing.....	14
1 Items to be Tested:.....	14
2 Test Specifications.....	14
3 Test Results.....	17
4 Regression Testing.....	18
IV Inspection.....	19
1 Items to be Inspected.....	19
2 Inspection Procedures.....	19
3 Inspection Results.....	22
V Recommendations and Conclusions.....	24
VI Project Issues.....	24
1 Open Issues.....	24
2 Waiting Room.....	26
3 Ideas for Solutions.....	26
4 Project Retrospective.....	27
VII Glossary.....	28
VIII References / Bibliography.....	29
IX Index.....	29

List of Figures

Figure 1: UML diagram from Release 1.....	10
Figure 2: UML/Class Diagram from Release 2.....	12

List of Tables

Table 1: Definitions of Key Terms.....	8
--	---

I Project Description

1 Project Overview

Art Exhibition Management System (AEMS), originally proposed by Patel et al. [1], is a web-based application designed to streamline the management of art exhibitions for organizers, securely and safely manage orders and auctions of artworks, and allow for customers to purchase and view artworks, tickets, participate in auctions and in exhibitions held by organizers.

2 Project Domain

Our project belongs to the domain of art exhibition management and digital museum/gallery services. Art galleries, museums, artists, and collectors often need a system to organize exhibitions, manage artwork information, sell or reserve tickets, support artwork purchases, and improve the visitor experience. Traditional exhibition management can involve manual tracking of artworks, schedules, visitors, sales, and events, which can lead to inefficiency, missing information, and difficulty analyzing audience engagement.

The Art Exhibition Management System, or AEMS, is designed to support both administrative users and customers. Administrators can manage artworks, exhibitions, customer information, orders, auctions, and reports. Customers can browse available artwork, view exhibition and event information, buy tickets, make payments, participate in auctions, and interact with artwork details, including 3D artwork viewing where available.

Understanding this domain is important for testing the project because the system must be evaluated based on how well it supports real art exhibition workflows. Correctness depends on whether users can successfully register and log in, view accurate artwork and exhibition data, complete ticket or artwork purchases, place auction bids, and whether administrators can maintain the system's content reliably. The value of the project comes from making art exhibitions more accessible, organized, and interactive for both visitors and gallery staff.

3 Relationship to Other Documents

The design report [1] was prepared by Group 2 students in Spring 2023. In that document, Patel et al. identified 5 product use cases:

1. Managing Art
2. Managing Customer
3. Managing Payment
4. Managing Order Update
5. Managing Order

Our prototype focuses on all given use cases , with some modifications based on the features we were able to implement in the final application. The original document also described related requirements such as artwork management, exhibition management, sales management, and reporting. These ideas were used as the foundation for our implemented system.

3a Use Case 1: Authentication

This use case identifies several objects:

- Original designation → Our designation
- Customers → Customer users
- System User → Admin users
- Super Admin → Admin users
- Login/Signup page → LoginPage and RegisterPage
- User session → ProtectedRoute and role-based navigation
- Art Exhibition Home Page → CustomerDashboard or AdminDashboard

Our implementation allows users to register and log in to the AEMS application. After authentication, the system directs users to the correct dashboard based on their role. Customer users are sent to the customer dashboard, where they can browse art, view events, buy tickets, access their cart, make payments, and participate in auctions. Admin users are sent to the admin dashboard, where they can manage artworks, exhibitions, orders, customers, auctions, and reports.

The original document mentioned email verification as part of authentication. Our current prototype implements registration, login, user roles, and protected routes, but does not fully implement email verification.

3b Use Case 2: Buy Exhibition Tickets

Objects identified by this use case:

Original designation → Our designation

Exhibition list → UpcomingEventsPage and BuyTicketsPage

- Customer → Customer user
- Payment Page → MakePaymentsPage
- Ticket purchase → Ticket/order workflow
- Exhibition information → Exhibition and ArchivedEvent records

Our implementation supports the ticket-purchasing workflow by allowing customer users to view upcoming events and exhibitions, navigate to the ticket page, select ticket-related options, and continue to payment. This connects to the original requirement that customers should be able to browse current exhibitions and purchase tickets through the system.

The original document also stated that tickets should be displayed and emailed to the user after payment. Our prototype supports the main user flow of viewing events, buying tickets, and making payments, but the automatic email ticket feature is not fully implemented.

3c Use Case 3: System User Management

Objects identified by this use case:

- Original designation → Our designation
- System User → Admin user
- Orders → AdminOrders
- Exhibitions → AdminExhibitions
- Inventory/artwork data → AdminArtworks and Artwork model
- Data management → AdminDashboard and AdminReports
- Saved work → Backend database records

Our implementation gives admin users access to management pages for the core exhibition system. Admins can manage artworks, exhibitions, customer orders, customer information, auctions, and reports. This matches the original use case where system users manage orders, exhibitions, inventory, and system data.

In the original document, system users were expected to send their work to a super admin for approval. In our prototype, the admin role directly manages the system content, so the separate approval workflow was simplified.

3d Use Case 4: Super Admin Management

- Objects identified by this use case:
- Original designation → Our designation
- Super Admin → Admin user
- Manage sales → AdminOrders and AdminReports
- Manage art → AdminArtworks
- Manage exhibitions → AdminExhibitions
- Post updates to home page → Customer-facing artwork and event pages
- Review system activity → AdminDashboard and reports

Our implementation combines the original system user and super admin responsibilities into the admin role. Admin users can add and edit artwork, manage exhibitions, view orders, review customer activity, manage auctions, and view dashboard/report data. These features support the original super admin goal of overseeing system content and making updates available to customers.

The original document separated system users and super admins into different approval levels. Our final prototype uses a simpler role structure with customers and admins, which was more practical for the scope of our implementation.

4 Naming Conventions and Definitions

4a Definitions of Key Terms

User	Any individual interacting with the system, including
------	---

	customers, administrative staff, and organizers.
Admin	An exhibition organizer responsible for managing exhibitions, artwork, and related data within the system.
Customer	A user who can browse exhibitions and purchase tickets or artwork through the system provided by admin users.
Artwork	A piece of art created by an artist, including associated information such as title and price.
Exhibition	A scheduled event where artworks are displayed to the public, including details such as date, location, and participating artists.
Auction	A scheduled sale where artworks are sold to the highest bidder, through bidding that increases the price.
Protected Route	A page in the application that requires a logged-in user and, in some cases, a specific role such as customer or admin.

Table 1: Definitions of Key Terms

4b UML and Other Notation Used in This Document

UML diagrams in this document follow UML Distilled, 3rd edition [2]. Any exceptions are noted where used.

4c Data Dictionary for Any Included Models

The data for our application is stored in a relational database using SQLite. SQLite is used with the Django backend to store and manage the main data models for the AEMS application. The full database schema is available in the project repository through the Django model files and migrations.

The main models used in our system include CustomUser, Artwork, Exhibition, Order, Auction, and Bid. These models support the major workflows tested in the project, including user authentication, artwork browsing, exhibition management, ticket purchasing, payment processing, order tracking, and auction bidding. Any included diagrams or figures in the appendix show the relationships between these database entities.

II Project Deliverables

This semester, our group produced a working prototype of the Art Exhibition Management System (AEMS). The system includes a React/TypeScript frontend and a Django backend using a SQLite database. The application supports user registration and login, role-based access for customers and administrators, customer artwork browsing, event viewing, ticket purchasing, cart/payment workflows, auctions, and administrator management pages.

Our project deliverables include the implemented web application, backend database models, customer and admin interfaces, scenario documentation, testing materials, and final report documentation. The prototype demonstrates the main workflows described in the original project document, especially customer interaction with exhibitions, artwork, tickets, and payments, as well as administrator management of system data.

1 First Release

For the first release, our group focused on Scenario 1: Login and Customer Interface Exploration. This release implemented the customer-facing portion of AEMS. Users could start at the welcome page, register or log in, and access the customer dashboard based on their assigned role.

As of the first release, the system supported the following functionality:

- Customer account creation and login
- Role-based navigation to the customer dashboard
- Viewing and buying artworks
- Viewing artwork details, including images, descriptions, prices, and 3D artwork viewing where available
- Viewing upcoming events and exhibitions
- Buying exhibition tickets
- Adding selected artworks to a cart
- Completing payment through the payment page
- Receiving confirmation after a purchase workflow

This release mainly addressed the original customer use cases from the AEMS project description, especially authentication, browsing exhibitions/artworks, buying tickets, and making purchases. Some planned features, such as sending digital tickets through email or mobile number, were identified as future improvements.

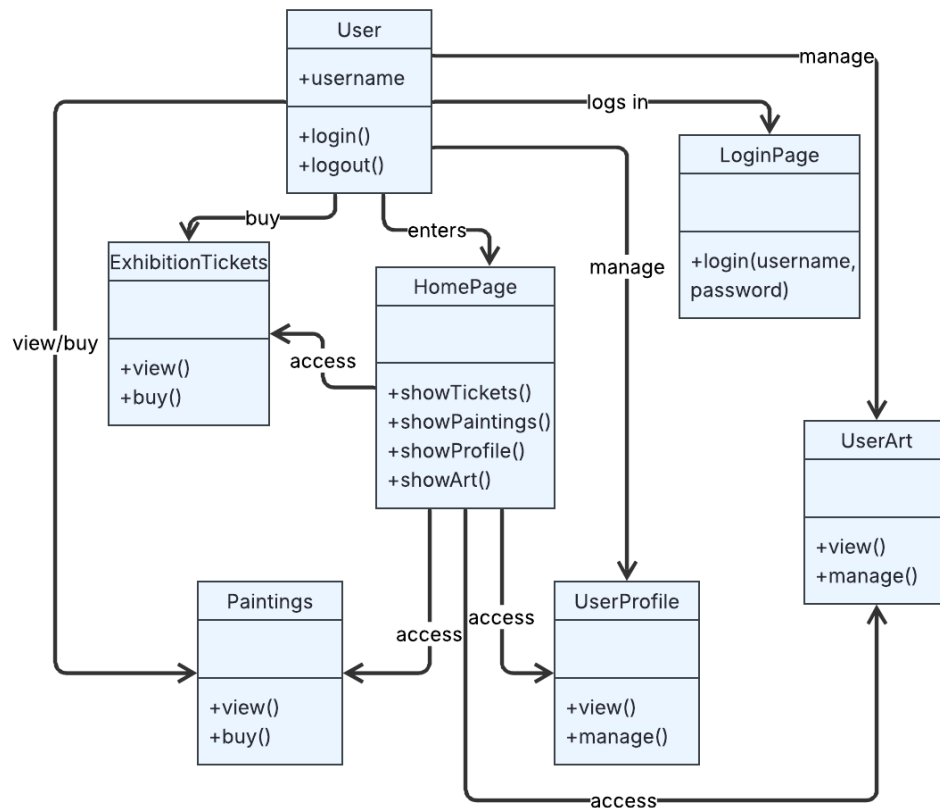


Figure 1: UML diagram from Release 1

Figure 1 shows the initial UML diagram developed for Release 1 of AEMS. The diagram focuses on the customer side of the application and shows how a user interacts with the main pages of the system. The User object can log in through the LoginPage, enter the HomePage, access their UserProfile, view or buy paintings, and buy ExhibitionTickets. The diagram also includes UserArt, which represents artwork-related management or viewing functionality.

2 Second Release

For the second release, our group focused on Scenario 2: Administrative Control, Live Art Bidding, and 3D Modeling. This release expanded AEMS beyond the customer interface from Release 1 and added administrator-side functionality, auction support, reporting features, and improved artwork viewing.

As of the second release, the system supported the following functionality:

- Admin login and role-based access to the admin dashboard.
- Admin management of artworks, including adding, editing, deleting, pricing, and status updates.

- Admin management of exhibitions, including creating, editing, and publishing exhibition details.
- Admin order management for viewing and updating customer orders
- Auction management, including selecting artwork, setting bid amounts, start/end times, and auction status.
- Customer auction participation through the customer auction page.
- Bid tracking using the Auction and Bid models.
- 3D artwork viewing for supported artwork models.
- Reports and analytics for orders, ticket sales, auctions, customers, and artwork performance.

This release addressed the original AEMS goals related to system user management and super admin management. It also extended the customer side by adding auction participation and 3D artwork viewing. The second release made the system more complete by allowing administrators to control the content and monitor system activity instead of only providing customer browsing and purchase features.

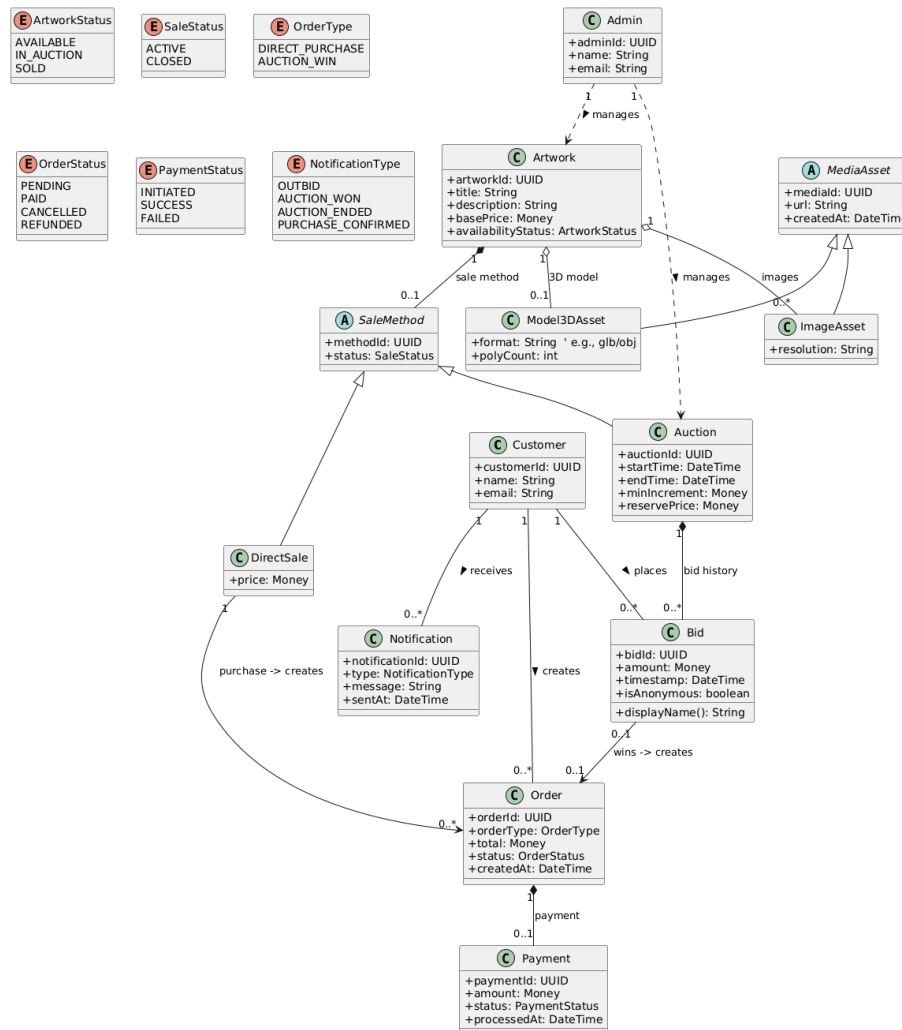


Figure 2: UML/Class Diagram from Release 2

Figure 2 shows the planned structure for Release 2, which focused on admin-side control and expanded artwork features. The diagram represents the relationship between users, admins, artworks, exhibitions, auctions, bids, and reports. During Release 2, we implemented the main admin dashboard, artwork management, exhibition management, order management, auction management, customer bidding, and reporting pages. Some planned features, such as automatic winner payment requests and report exporting, were not fully completed and remain possible future improvements.

3 Third Release

For the third release, our group focused on Scenario 3: 3D Models, Pipeline Refinement, and Full Analytic Displays. This release improved the connection between customer actions and admin-side records. It also refined existing pages for reports, exhibitions, transactions, profile history, and artwork viewing.

As of the third release, the system supported the following functionality:

- Improved admin dashboard with summary statistics
- Reports and analytics for orders, ticket sales, auctions, customers, revenue, and artwork performance
- Admin visibility into recent orders and customer activity
- Customer profile page with purchase/order history
- Improved customer artwork browsing and 3D artwork viewing
- More detailed upcoming event and exhibition displays
- Continued ticket and payment workflows connected to order records

Stronger connection between customer purchases and admin reports

This release addressed the original AEMS goals related to reporting, customer interaction tracking, exhibition display, and immersive artwork viewing. Some planned features, such as real-time report updates, support ticket requests, report exporting, and full multi-artist exhibition support, were not fully completed and were documented as future improvements.

4 Comparison with Original Project Design Document

The prototype produced compares with the full project described by the previous group where the prototype included a wide range of the core components, whilst some features were not implemented. In the project description, the previous group's provided requirements that were met with the prototype are user authentication, accounts, ticket purchasing for exhibitions, artist management, artwork management, exhibition management, reporting and analytics. The prototype successfully implemented the core features and functionalities that the previous group had described, however, there were some aspects that were not implemented. For

example, the previous group's design describes the implementation of a "Super Admin" user type, which is not included in the prototype. In addition, the structure and system design of the prototype is less complex and scalable than the more robust description from the full project. The prototype's structure is primarily a monolith with minimal microservices that is meant for demonstrating functionality, and the prototype also does not fully support the functional requirements of high performance or security measures compared to the project description.

III Testing

1 Items to be Tested:

1. Customer Portal
 - a. View/Buy Arts API endpoints - IID #01
 - b. View/Buy Arts 3D renderer - IID #02
 - c. Art Purchasing - IID #03
 - d. Events API endpoints - IID #04
 - e. Events Ticket Purchasing - IID #05
 - f. Cart Page - IID #06
 - g. Profile Page - IID #07
 - h. Settings - IID #08
 - i. Account Creation - IID #09
2. Admin Portal
 - a. Reports - IID #10
 - b. New Art Registration - IID #11
 - c. New Auction Registration - IID #12
 - d. Orders Page - IID #13
 - e. Exhibition/Events Page - IID #14
 - f. Customers Database - IID #15

2 Test Specifications

ID#1 - Account Creation

Description: Ability to create an account and have it stored in a backend database.

Items covered by this test: IID #09, #15

Requirements addressed by this test: UC #01

Environmental needs: SQLite, Django, Vite, Node.js

Intercase Dependencies: N/A

Test Procedures: Run commands: npm run dev, and python manage.py runserver. Then click login, and account creation. Insert values, and then return to login and use login details created.

Input Specification: Password created must contain at least 6 characters and 1 special character. Email must have @, and domain.

Output Specifications: SQLite database updates with new user account info.

Pass/Fail Criteria: SQLite database successfully stored the new user details, and anyone can use the user and password to login.

ID#2 - View/Buy Arts Page

Description: When viewing the View/Buy Arts tab, the user should be able to see art from the Art Institute of Chicago API.

Items covered by this test: IID #01

Requirements addressed by this test: UC #01

Environmental needs: SQLite, Django, Vite, Node.js

Intercase Dependencies: ID #1

Test Procedures: Run commands: npm run dev, and python manage.py runserver. Then login with the customer account. Then click on View/Buy Arts Button. Django backend should say it has sent a request to Art Institute of Chicago API to retrieve artworks that are viewable on that page.

Input Specification: Mouse click on the view arts button is required.

Output Specifications: 10,000+ viewable art cards with prices, summaries, and pictures shown.

Pass/Fail Criteria: To pass the test, Django successfully retrieves the data from Art Institute of Chicago API and makes them viewable to the customer. It fails if the list of art pieces is empty, and the customer user is unable to view any of the artworks on the View/Buy Arts page.

ID#3 - Buying Art

Description: The customer clicks on an available listed art piece and buys it, sending it to their cart.

Items covered by this test: IID #01, #03, #06, #07

Requirements addressed by this test: UC #03

Environmental needs: SQLite, Django, Vite, Node.js

Intercase Dependencies: ID #1, #2

Test Procedures: Run commands: npm run dev, and python manage.py runserver. Then login with the customer account. Then click on View/Buy Arts Button. Customers should click on an art piece and click add to cart.

Input Specification: Mouse click on the add to cart button.

Output Specifications: Cart page should show all added listed artworks by customer and their price, and amount. The checkout page should allow users to enter credit card details.

Pass/Fail Criteria: To pass the test, the backend database should record the purchase to be sent to the admin portal. It should be viewable in the customer's profile page.

ID#4 - Events Page Buying and Viewing

Description: The customer clicks on the events tab, to see a list of upcoming events. The admin can create events as well.

Items covered by this test: IID #04, #05, #14

Requirements addressed by this test: UC #02

Environmental needs: SQLite, Django, Vite, Node.js

Intercase Dependencies: ID #1

Test Procedures: Run commands: npm run dev, and python manage.py runserver. Then login with the customer account. Click the events tab and there should be events there.

Input Specification: Mouse click on the events button.

Output Specifications: Events tab should have events archived from SerpAPI and Art Institute of Chicago API. These events should be ordered from earliest to latest.

Pass/Fail Criteria: To pass the test, the backend database should record the retrieved data from Serpapi so it does not have to make any future calls. Art Institute of Chicago API can be called freely. Admin should be able to add events so that those events can be seen by customers.

ID#5 - Admin 3D Art Creation

Description: The admin portal allows the creation of art that includes a 3D rendered model that can be viewed by the customer.

Items covered by this test: IID #02, #11

Requirements addressed by this test: UC #04

Environmental needs: SQLite, Django, Vite, Node.js

Inter-case Dependencies: ID #1

Test Procedures: Run commands: npm run dev, and python manage.py runserver. Then login with the admin account. Get to the art tab to insert a picture of art and upload a 3D model.

Input Specification: Mouse click on the arts tab in admin portal.

Output Specifications: Should reveal an art profile registered with a viewable 3D model when viewed from the customer in the View/Buy Arts tab.

Pass/Fail Criteria: The test fails if the customer is unable to view the newly created art model from the admin. It also fails if the art model does not pan correctly when viewing.

3 Test Results

ID#1 - Account Creation

Date(s) of Execution: 3/18

Staff conducting tests: Abrar, Aditi, Brian

Expected Results: Customer and Admin accounts are registered into the database.

Actual Results: Successfully registered customer and Admin into database.

Test Status: Pass

ID#2 - View/Buy Arts Page

Date(s) of Execution: 4/4

Staff conducting tests: Aditi, Abrar

Expected Results: Art Institute of Chicago API retrieves art data and shows it.

Actual Results: API showed most relevant results including pictures.

Test Status: Pass

ID#3 - Buying Art

Date(s) of Execution: 4/12

Staff conducting tests: Brian, Aditi

Expected Results: The checkout page should list all the purchases made by the customer.

Actual Results: Checkout page shows all the purchases made by a customer in their profile.

Test Status: Pass

ID#4 - Events Page Buying and Viewing

Date(s) of Execution: 4/12

Staff conducting tests: Abrar, Brian

Expected Results: Events page should archive serpapi retrieved events, and Art Institute of Chicago API retrieved events to be bought in checkout.

Actual Results: Checkout page shows the purchases for the tickets for events correctly, and the events page correctly shows all events from both APIs.

Test Status: Pass

ID#5 - Admin 3D Art Creation

Date(s) of Execution: 4/25

Staff conducting tests: Aditi, Brian

Expected Results: 3D render tab should let the user pan the 3D model of the specific art exhibition.

Actual Results: The user can pan and view the 3D art model.

Test Status: Pass

4 Regression Testing

Regression testing was conducted throughout development to ensure that newly added features did not break existing functionality.

Key regression tests included:

- Re-testing login and authentication after implementing authorization fixes

- Re-testing artwork uploads after modifying file validation logic
- Re-testing auction bidding after updating bid validation rules
- Re-testing events page after improving API error handling

Results:

All previously working features continued to function correctly after updates. No major regressions were identified.

IV Inspection

1 Items to be Inspected

List of items to be inspected:

1. User Authentication and Authorization
2. Events Page and Exhibitions Display
3. 3D Artwork Rendering and Uploading
4. Backend Media Management and Artwork Uploading
5. Auction & Bidding Workflow
6. Exhibition Scheduling and Display

2 Inspection Procedures

The inspection procedure for any items above should start and follow the checklist template below.

Inspection#ID

Item to be inspected:

Inspector:

What is expected:

What is delivered feedback:

Inspection#01

Item to be inspected: User Authentication and Authorization

Inspector: Brian Daza

What is expected: Users can register, login, and reset passwords. Passwords are stored in the backend and successfully hashed to ensure security. The API endpoints and pages require authentication and authorization.

What is delivered feedback: The registration and login flows functioned correctly during testing. Passwords were successfully stored in the backend using hashing, ensuring that plain text passwords were not exposed. Authentication tokens were generated and maintained session persistence as expected.

Inspection#02

Item to be inspected: Events Page and Exhibitions Display

Inspector: Brian Daza

What is expected: The events page displayed exhibitions and events as expected, with relevant images and information successfully retrieved from the SerpAPI. The layout was responsive and visually consistent across different screen sizes.

What is delivered feedback: The events page displayed exhibitions and events as expected. Some issues were observed during testing. In cases of slow or failed API responses, the page did not gracefully handle errors, resulting in empty sections without user feedback. Additionally, duplicate or outdated event entries occasionally appeared due to inconsistent filtering of API data.

Inspection#03

Item to be inspected: 3D Artwork Rendering and Uploading

Inspector: Aditi Ghosh

What is expected: The 3D viewer renders supported model formats correctly, and the upload workflow validates file types and sizes for the customer users. The thumbnails are also able to be uploaded when creating the artwork.

What is delivered feedback: The 3D viewer rendered the sample models correctly, and the upload process accepted 3D model files but did not have file size validation. The viewer also only supports .glb files and not other files such as .fbx files.

Inspection#04

Item to be inspected: Artwork Uploading and Media File Management

Inspector: Aditi Ghosh

What is expected: Artwork upload forms accept metadata (title, description, artist, price) along with images and validate file types and sizes. Uploaded files should be stored correctly and retrievable without errors.

What is delivered feedback: The upload forms successfully accepted artwork metadata and associated images. Images were stored and retrieved correctly for display in the dashboard. However, file type validation was partially implemented, and file size restrictions were not enforced, allowing large files to be uploaded without warning. No critical failures occurred, but improvements are needed to prevent potential storage and performance issues. A re-inspection is recommended after validation enhancements are implemented.

Inspection#05

Item to be inspected: Auction & Bidding Workflow

Inspector: Abrar Makki

What is expected: The bidding workflow accepts valid bid amounts and ensures that each newly created bid is higher than the current highest bid. Only authenticated and authorized customer users may place bids, and the closing of bids notifies the winner and prevents further bids.

What is delivered feedback: The bids were recorded and the basic bid flow worked; however, the frontend currently allows bidding without enforcing authentication in some cases. Additionally, validation to ensure that new bids are strictly higher than the current highest bid was inconsistently enforced on the client side. Backend validation worked but needs stronger frontend integration. Notifications for auction closure were not fully implemented. A re-inspection is required after authentication checks and validation logic are fully enforced.

Inspection#06

Item to be inspected: Exhibition Scheduling and Display

Inspector: Abrar Makki

What is expected: Exhibition entries include correct start and end times, and the upcoming filtering excludes past exhibitions. The exhibition thumbnails should show accurate information, and any edits made by admin users should reflect correctly in real time or after refresh.

What is delivered feedback: Exhibitions displayed correctly with accurate titles, images, and scheduling information. The filtering of upcoming versus past exhibitions worked as expected. Admin edits were reflected after page refresh, though not always in real time. No major flaws were identified, but implementing live updates or auto-refresh functionality would improve usability. No re-inspection was deemed necessary at this stage.

3 Inspection Results

The following section summarizes the results of the inspections performed on the major AEMS features. Each inspection records what was reviewed, who inspected it, what flaws were found, and whether the issue was resolved or should be re-inspected in a future version.

Inspection #01: User Authentication and Authorization

Inspector: Brian Daza

Item inspected: User registration, login, password storage, authentication, and authorization.

Result: The registration and login flows worked correctly during testing. Passwords were stored in the backend using hashing, so plain text passwords were not exposed. Authentication tokens were generated successfully, and session persistence worked as expected.

Flaws discovered: No major flaws were found during this inspection.

Resolution: This item passed inspection. No re-inspection was required for the current release.

Inspection #02: Events Page and Exhibitions Display

Inspector: Brian Daza

Item inspected: Events page, exhibition display, images, event information, SerpAPI data retrieval, and responsive layout.

Result: The events page displayed exhibitions and events as expected. Relevant images and event information were retrieved and shown in the interface. The layout was visually consistent and responsive across different screen sizes.

Flaws discovered: Some issues were found with API error handling. When the SerpAPI response was slow or failed, the page sometimes displayed empty sections without giving feedback to the user. Duplicate or outdated event entries also appeared occasionally because API data filtering was inconsistent.

Resolution: The main events page functionality was accepted, but error handling and duplicate filtering should be improved. A future re-inspection is recommended after API fallback messages and stronger event filtering are added.

Inspection #03: 3D Artwork Rendering and Uploading

Inspector: Aditi Ghosh

Item inspected: 3D artwork viewer, supported model formats, upload workflow, file validation, and artwork thumbnails.

Result: The 3D viewer rendered the sample models correctly. The upload process accepted 3D model files, and thumbnails could be uploaded when artwork was created.

Flaws discovered: The upload workflow did not enforce file size validation. The 3D viewer currently supports .glb files, but does not support other model formats such as .fbx.

Resolution: The 3D viewer was accepted as functional for supported .glb models. File size validation and support for additional model formats should be added in a future version. A re-inspection is recommended after these improvements are implemented.

Inspection #04: Artwork Uploading and Media File Management

Inspector: Aditi Ghosh

Item inspected: Artwork upload forms, metadata fields, image upload, file validation, file storage, and dashboard retrieval.

Result: The artwork upload forms successfully accepted metadata such as title, description, artist, and price. Associated artwork images were stored and retrieved correctly for display in the dashboard.

Flaws discovered: File type validation was only partially implemented, and file size restrictions were not enforced. This means large files could be uploaded without warning, which may cause storage or performance issues in future versions.

Resolution: No critical failures occurred, so the feature was accepted for the current release. Validation improvements are still needed, and a re-inspection is recommended after file type and file size restrictions are implemented.

Inspection #05: Auction and Bidding Workflow

Inspector: Abrar Makki

Item inspected: Customer bidding workflow, bid amount validation, authentication checks, auction closing, and winner notification.

Result: Bids were recorded successfully, and the basic bidding workflow worked during testing. Backend validation was present and helped protect the bid data.

Flaws discovered: The frontend allowed bidding without enforcing authentication in some cases. Client-side validation did not always ensure that a new bid was strictly higher than the current highest bid. Notifications for auction closure and winner selection were not fully implemented.

Resolution: The basic auction workflow was accepted as a prototype feature, but the feature requires more work before production use. Authentication enforcement, frontend bid validation, and auction closure notifications should be improved. A re-inspection is required after these changes are made.

Inspection #06: Exhibition Scheduling and Display

Inspector: Abrar Makki

Item inspected: Exhibition start/end times, upcoming exhibition filtering, thumbnails, displayed information, and admin edits.

Result: Exhibitions displayed correctly with accurate titles, images, and scheduling information. The filtering between upcoming and past exhibitions worked as expected. Admin edits appeared correctly after refreshing the page.

Flaws discovered: Admin edits were not always reflected in real time.

Resolution: No major flaws were identified, and no re-inspection was required for the current release. Live updates or auto-refresh functionality would improve usability in a future version.

V Recommendations and Conclusions

The majority of the items and core features function, however, several important issues were found in security, file handling, and accessibility. The recommended next steps are to implement permission checks, add file type and size validation as well as support for more 3D file types, and improve accessibility. Further testing is also recommended, particularly to prioritize fixes that affect security and data integrity as these are imperative to the dashboard's functionality.

VI Project Issues

1 Open Issues

Several issues remain open for the AEMS project and may affect future versions of the product.

One open issue is the difference between the original project scope and the final prototype scope. The original document described a larger art exhibition management system with advanced features such as email verification, digital ticket delivery, inventory management, sales tracking, analytics, and possible virtual reality support. Our prototype implements the main customer and admin workflows, but some of the more advanced features were simplified or left incomplete due to time constraints.

Digital Ticket Delivery

Another unresolved issue is the ticket confirmation process. The original scenario described sending digital tickets to the customer by email or phone after purchase. In the current prototype, the user can complete the ticket/payment workflow, but automatic ticket delivery through email or SMS is not fully implemented. This would need additional backend integration and testing in a future release.

Real Payment Processing

A third open issue is payment processing. The application includes a payment page and order creation flow, but it does not connect to a real payment provider. Because of this, payment testing is limited to the prototype workflow instead of real financial transaction validation. If the system were deployed publicly, secure payment processing, error handling, refunds, and transaction records would need to be completed.

Auction Completion and Real-Time Bidding

Auction handling is also an open issue. The system includes auction and bid functionality, but future work is needed for automatic auction closing, winner selection, payment request generation, and stronger real-time bid updates. These features are important because live bidding requires accurate timing and reliable bid tracking.

Reporting and Analytics Limitations

Finally, some reporting and analytics features are still limited. The admin dashboard and reports page provide useful summary information, but future work could include exportable reports, deeper filtering, real visitor engagement tracking, and more accurate sales/ticket analytics. These improvements would help the system better match the goals described in the original project document.

2 Waiting Room

Several valuable ideas were proposed by Patel et al. [1] and some were identified by our team during development as useful extensions to the system that could not be included in the current release due to time constraints, technical limitations, or dependency on external systems. These requirements remain important for future versions of the application and are documented here to prevent their loss.

1. **WR-01: Email or SMS Ticket Delivery:**

A future version of AEMS should send digital tickets to customers through email or text message after a successful ticket purchase. This was part of the original project idea, but it was not included in the current release because it requires additional notification services and backend integration.

2. **WR-02: Real Payment Gateway Integration:**

The current payment workflow is a prototype. A future release should connect the system to a real payment provider so customers can securely purchase tickets and artwork. This would also allow the system to support refunds, transaction IDs, and stronger payment validation.

3. **WR-03: Automatic Auction Winner Selection:**

Future versions should automatically close auctions when the end time is reached, select the highest bidder, update the artwork status, and generate a payment request for the winner. This would make the auction feature more complete and reliable.

4. **WR-04: Advanced 3D or Virtual Reality Exhibition View:**

The original AEMS idea included a more immersive art-viewing experience. A future version could expand the current 3D artwork viewer into a full virtual gallery or VR-style exhibition walkthrough.

5. **WR-05: Artwork Recommendations**

A future release could recommend artworks or exhibitions to customers based on their browsing history, purchases, favorite categories, or auction activity. This would make the app more personalized.

6. **WR-06: User Reviews and Ratings**

Customers could be allowed to leave ratings or comments on artworks, exhibitions, or events. This would help increase visitor engagement and provide useful feedback to admins and artists.

3 Ideas for Solutions

1. **Continue Using React and Django:**

AEMS should continue using React for the frontend and Django for the backend because these tools already support the current prototype. React works well for building interactive pages such as dashboards, artwork galleries, carts, auctions, and reports. Django is useful for handling user authentication, database models, API endpoints, and admin-side data management.

2. Use Django REST Framework for API Development:

Future backend features should continue to use Django REST Framework so the frontend can communicate with the backend through organized API endpoints. This would be useful for features such as ticket records, auction bids, payment history, artist profiles, and report data.

3. Migrate from SQLite to PostgreSQL for Deployment:

SQLite is suitable for development and testing, but a production version of AEMS should use PostgreSQL. PostgreSQL would better support larger datasets, multiple users, concurrent orders, and live auction activity.

4. Add a Real Payment Provider:

A future version should integrate a secure payment service such as Stripe or PayPal. This would allow the system to process real ticket and artwork purchases, validate card information, generate transaction IDs, and support refunds.

5. Add Email or SMS Notification Services:

To support digital ticket delivery and order confirmations, the system could use email or SMS services. For example, the backend could send confirmation emails after ticket purchases, artwork orders, or auction wins.

6. Use WebSockets for Live Auctions:

The auction feature could be improved by using WebSockets so customers and admins can see live bid updates immediately. This would make auctions feel more realistic and reduce the need to refresh the page.

7. Expand 3D Artwork Viewing:

The existing 3D artwork viewer could be expanded by supporting more 3D model formats and a virtual gallery view so that the users can have an experience of watching an exhibition in a real environment. This would bring the system closer to the immersive experience described in the original AEMS document.

4 Project Retrospective

At the conclusion of this project, what went well for the group was the weekly scrum meetings. Every week, the group members all met in person to discuss each others' work accomplishments, contributions, issues, and future implementations. These allowed not only each member to be well informed and communicative of the

project's progress, but the meetings also prevented any requirements being overlooked, group members' contributions overlapping, and any shortcomings in deliverables. In addition, some of the members also performed pair programming, where a member would implement and test some of the features on the workspace, whilst the other would review and watch for errors and bugs. These occasional instances of pair programming worked very well not only for preventing bugs and errors, but also worked in ensuring both members had a good grasp of the project's functionality. Furthermore, the group also made use of the Jira KanBan methods that kept the group tasks in check as well as help indicate prioritized features and updates which worked very well. However, what did not work as well was time management and feature prioritization. Some features, such as the admin page statistics, reports, and auction functionalities, were started late in development and could not be fully completed to the desired extent. Additionally, testing was sometimes performed after implementation rather than alongside it, which led to minor bugs being discovered later than expected. In the future, the team should adopt a stricter development schedule with earlier testing integration, better task distribution, and clearer prioritization of core features over optional enhancements. Some suggestions and improvements that should be made is the inclusion of test driven development, as throughout the development of the project, some features and functionalities became temporarily dysfunctional and created minimal errors as a result of newly implemented features. Having utilized the method of test driven development would have greatly improved the quality of the added functionality and code, and would further assist in preventing errors in future development.

VII Glossary

User: Any individual interacting with the system, including customers, administrative staff, and organizers.

Admin: An exhibition organizer responsible for managing exhibitions, artwork, and related data within the system.

Customer: A user who can browse exhibitions and purchase tickets or artwork through the system provided by admin users.

Artwork: A piece of art created by an artist, including associated information such as title and price.

Exhibition: A scheduled event where artworks are displayed to the public, including details such as date, location, and participating artists.

Auction: A scheduled sale where artworks are sold to the highest bidder, through bidding that increases the price.

Bid: An offer placed by a customer during an auction. Bid records store the auction, user, amount, timestamp, and anonymous bidding option.

3D Artwork Viewer: A customer-facing feature that displays supported 3D artwork models, mainly .glb files, for a more interactive viewing experience.

VIII References / Bibliography

IX Index

- [1] J. Patel et al. “AEEMS.” University of Illinois Chicago, Chicago, IL, USA. Group 2, Spring 2023.**
- [2] M. Fowler. UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. New York, NY, USA: Addison-Wesley Professional, 2003.**